Architecture

This document describes the system we are building and how its parts fit together. It reflects the decisions recorded in `DECISIONS.md` (ADR-001…013). It describes the intended design; sections marked **(planned)** are not yet built. See `roadmap.md` for current status.

> Note: `docs/proj_desc.md` is an AI-generated idea list, not an authoritative spec. Decisions here are justified on production-engineering grounds (security, auditability, a minimal trusted core, testability), not by spec compliance.

---

1. What the system does

A user gives the platform an Android APK they are authorized to assess. The platform:

1. Validates and stores the file safely.
2. Statically analyzes it (no code execution) to find what the app is made of: manifest facts, permissions, embedded URLs, keys, SDKs, and where each was found.
3. Builds a model of the app's attack surface: the backends and resources it appears to talk to, each traceable to the evidence that suggested it.
4. Lets a bounded agent form security hypotheses, choose checks from a fixed toolbox, and, only within an authorized scope, run safe checks against the backend.
5. Verifies each result with deterministic rules and records findings backed by evidence.

The central principle:

> **Discover through analysis, investigate through reasoning, validate through controlled experiments, and report only what the evidence supports.**

A public client key is not a vulnerability. A backend that returns records it should not is. The system is built to tell those two apart, and to say "not assessed" when it cannot tell.

---

2. The one idea that shapes everything: the LLM proposes, code decides

A weak version of this project is "an LLM that calls security tools." The design instead puts hard, deterministic boundaries around the model:

- The LLM **reasons and proposes**: it interprets evidence, forms hypotheses, and picks the next action as a *registered tool name plus arguments*.
- Plain, testable code **decides and proves**: a scope gate authorizes (or blocks) every action, tools enforce their own limits, and a separate verifier judges the real output.
- The LLM never runs shell commands or code, never grants itself permission, and its claim that something worked is never the proof that it did.

Everything below is in service of that split.

---

3. Trust zones

The strongest boundary in the system is: **credentials and hostile files never share a zone.** Three zones (ADR-003, ADR-005):

```
 Zone 1: APK SANDBOX                Zone 2: ORCHESTRATOR            Zone 3: LLM
GATEWAY
 (sandbox-worker, sandbox-db)       (api, worker, main-db)         (llm-gateway)
 - opens/decompiles hostile APKs    - agent loop                   - holds Claude
creds
 - no internet egress               - scope gate, tool registry    - text in, JSON
out
 - no credentials                   - verifier, findings           - tools disabled
 - reaches only sandbox-db          - only service on both nets    - never sees raw
APKs
```

- The **sandbox** processes untrusted files and can reach only its own throwaway database. If an APK ever exploited a parser, there is nothing there worth stealing and no route to the real data.
- The **orchestrator** holds the real data and runs the agent. It is the only component attached to both the main and sandbox networks, and it treats everything coming back from the sandbox as untrusted input to be validated.
- The **LLM gateway** is the only place the Claude credential lives. It accepts a structured request, runs the Claude CLI with all built-in tools disabled, validates the output against a JSON schema, and returns it. It never touches APK files.

---

4. Components

Component	Zone	Responsibility
`frontend`	—	Server-rendered Jinja pages + compiled Tailwind; live agent trace via SSE + htmx. Served by the API.
`api` (FastAPI)	2	REST API, renders the dashboard, enqueues assessments, streams progress.
`worker` (Procrastinate)	2	Runs the assessment pipeline and the agent loop. Bridges to the sandbox.
`sandbox-worker`	1	Runs androguard + JADX on APKs. No internet, no credentials.
`main-db` (Postgres)	2	All platform data + the main job queue.
`sandbox-db` (Postgres)	1	Sandbox job queue + raw analysis output only. Untrusted, disposable.
`llm-gateway`	3	Claude CLI wrapped as prompt-in/JSON-out, tools disabled.
storage (host dir)	2/1	APKs and evidence files, behind a storage interface.

All run under one `docker compose up` (ADR-001), ports bound to `127.0.0.1` only (ADR-007).

---

5. The assessment pipeline

Each stage is persisted (ADR-004a). A crash never leaves an assessment falsely “completed”; it resumes or fails explicitly.

```
APK upload
  -> validate (size, real zip, sha256, no zip-bomb, no path traversal)
  -> hand off to sandbox (outbox pattern, idempotent) ──► sandbox analyzes
  -> read & validate raw sandbox output                ◄──
  -> technology & endpoint discovery (rules)
  -> attack-surface model (resources + provenance)
  -> [agent loop begins]
  -> report
```

Sandbox handoff (outbox pattern, ADR-005)

Because the sandbox has its own database, we cannot enqueue its job in the same transaction as the main state update. Instead:

1. In `main-db`, one transaction records “analysis requested” and enqueues a main-db handoff job.
2. The handoff job inserts the sandbox job into `sandbox-db`, keyed by artifact id, so a retry after a crash is a no-op (idempotent).
3. When the sandbox finishes, the worker reads the raw result, **validates it as untrusted input**, and writes the trusted version into `main-db`.

---

6. The agent loop

The loop runs in the worker. Each iteration is one persisted “step” visible in the trace UI.



Stop conditions (ADR-011): step cap reached, request cap reached, no useful action remains, scope/approval unavailable, cancellation, or a safe-execution failure. A completed loop means “the process finished,” never “the app is secure.”

Why the gate and verifier are separate from the LLM

- The **scope gate** is the only thing that authorizes an action. The LLM cannot widen scope; a prompt injection hidden in decompiled code or an API response cannot either, because the gate reads the assessment's scope config, not the model's output.
- The **verifier** judges the actual tool output with fixed rules. The model's own "I confirmed it" is never the proof. This is what keeps findings honest.

We test both with adversarial cases: a forced out-of-scope action must be blocked, and a forced "it's vulnerable" claim with a denial response must be downgraded.

---

7. Generality: how it handles any app

Static analysis and hypothesis-forming work on any APK. Active checks come from **backend modules**. For the prototype only the **Supabase** module exists. On an app with a backend we have no module for (say Firebase), the agent still maps it and forms hypotheses, then honestly reports **"not assessed — no checks installed."** Adding a backend later means adding detection rules, a few registered tools, and verifier rules; the loop itself does not change.

---

8. The LLM gateway in detail (ADR-002, ADR-003)

- Backed by the local Claude Code CLI, validated in `spikes/001-claude-code-cli-as-llm.md`.
- Always invoked with `--tools ""`, `--strict-mcp-config`, `--setting-sources ""`, `--no-session-persistence`, `--json-schema <schema>`, from an empty temp dir. These flags are hardcoded in the gateway; callers cannot change them.
- Input is `{prompt_id, prompt_version, input_data, json_schema}`. The gateway builds the command; the caller never supplies raw CLI arguments.
- Output is validated against the schema. Malformed output is an **error**, never silently repaired into some other action.
- Logs token usage, cost, and duration — never prompt contents or secrets.
- The interface is provider-independent: swapping to an API key or a local model later replaces only the gateway's backend.

---

9. Data model (main-db)

Relational schema with Alembic migrations (ADR-004). Core entities, trimmed to what the prototype needs:

- **Project** — a unit of work owned by the (single) operator.
- **ApplicationArtifact** — uploaded APK: original filename, internal storage key, sha256, size, platform, package id, version, upload status.
- **Assessment** — scope_configuration, status, current_stage, timestamps, failure_reason.
- **AttackSurfaceResource** — type, identifier, source_location, discovery_method, confidence, **authorization_status** (discovered resources start *unapproved*), metadata.
- **Hypothesis** — claim, supporting evidence ids, missing_information, status, priority.
- **ToolExecution** — one agent step: tool_name, validated_arguments, status, timestamps, normalized_result, error_category. (Secrets are never stored here in the clear.)
- **Evidence** — type, storage_key, sanitized_metadata, content_hash, retention.
- **Finding** — title, severity, status (candidate/confirmed/inconclusive/not_assessed/ false_positive/…), the three dimensions (evidence strength, impact, confidence), impact, remediation, with provenance back to the ToolExecution and Evidence.
- **Settings** — key/value tunables, UI-editable (ADR-012).

Large files (APKs, raw evidence) live in storage, not in the database — only metadata and secure references are stored (ADR-004c).

Settings precedence (ADR-012)

`.env` seeds defaults → `settings` table overrides (global, UI-editable) → `scope_configuration` overrides per assessment. Settings hold tunable **limits** only, never **authorization**; scope stays in the assessment's scope config.

---

10. Safety controls

- **Artifact safety:** size limit, real-zip check, decompression ceiling, path-traversal rejection, sha256, never trust the extension. Files stored under internal keys.
- **Sandbox isolation:** non-root, no internet, resource limits, disposable workspace, own throwaway database.
- **Network scope (planned, Supabase phase):** every active request validates scheme, host, resolved IP, redirect target, and port against scope. Loopback/private/link-local blocked by default; the local lab is reachable only via an explicit per-assessment "authorized lab target" exception (ADR-001). Mitigates SSRF/DNS-rebinding.

- **Data minimization:** read-only, minimal rows, redacted samples; never bulk-pull.
- **Prompt-injection resistance:** decompiled code, strings, and API responses are untrusted data. Policy lives outside the model; the gate and verifier do not read model text as instructions.
- **Secrets:** redacted in logs and reports; never committed; `.env` and `/data` are git-ignored.

---

11. Local-only posture (ADR-001, ADR-007)

Single trusted operator, no login. Services bind to `127.0.0.1`. Because a browser on the same machine can still reach localhost, the API enforces a Host-header allowlist, strict CORS, and a custom-header requirement on state-changing requests (anti-CSRF). Ownership columns are kept where cheap so multi-user is a later extension, not a rewrite — but the prototype is explicitly **not** multi-user or deployable as-is.

---

12. Observability

Structured JSON logs and a trace id threaded from API request through worker and tool execution. Clear separation of error kinds: application, infrastructure, LLM, tool, scope denial, and finding. **A finding is not a failed tool run**, and the two are never conflated in logs or UI.